

LC 2577

Minimum time to visit a cell in a grid

Atlassian, Google

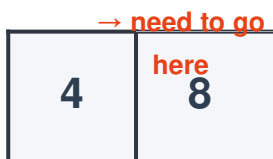
Core Concepts

- a cell can only be visited if $\text{currtime} \geq \text{cellvalue}$
- moving bw cell values makes $t++$;
- we need to go from (0,0) to (n-1, m-1)
- If $\text{currtime} \geq \text{cellvalue} \rightarrow$ visit immediately

Case 1

(for $\text{currtime} < \text{cellvalue}$)

Eg:



then arrival time = 5

diff = $(8 - 5) = 3$

So to waste 3 seconds it can go back and forth like:

[moving previous cell back and forth]

Observation

we can only waste even number of seconds ,

If odd number of seconds we will be gone somewhere else so we cant waste 1, 3, 5, 7 ... seconds alone , so we need +1 second to come back

Edge Case

0	99	
99		

→ **stuck,**

cant move neither can waste time as there's no previous cell where it can go back and forth

Implementation Code

```
class Solution {
public:
    int minimumTime(vector<vector<int>>& grid) {

        int n=grid.size();
        int m=grid[0].size();
        vector<pair<int,int>> dir={{-1,0},{0,-1},{1,0},{0,1}};

        if(grid[0][1]>1 && grid[1][0]>1) return -1;

        priority_queue<pair<int,pair<int,int>>,vector<pair<int,pair<int,int>>>,greater<>>>
        vector<vector<int>> dist(n,vector<int>(m,INT_MAX)); //here we will store

        pq.push({0,{0,0}});
        dist[0][0]=0;

        while(!pq.empty()){
```

```

        int r=pq.top().second.first;
        int c=pq.top().second.second;
        int currtime=pq.top().first;
        pq.pop();
        if(currtime>dist[r][c]) continue;
        if(r==n-1 && c==m-1) return currtime;

        for(auto &it:dir){
            int nr=r+it.first;
            int nc=c+it.second;

            if(nr>=0 && nc>=0 && nr<n && nc<m){
                int nexttime=currtime+1;

                //waiting logic
                if(nexttime<grid[nr][nc]){
                    if((grid[nr][nc]-nexttime)%2==0){
                        nexttime=grid[nr][nc];
                    }

                    else{
                        nexttime=grid[nr][nc]+1;
                    }
                }

                if(nexttime<dist[nr][nc]){
                    dist[nr][nc]=nexttime;
                    pq.push({nexttime,{nr,nc}});
                }
            }
        }

        return -1;
    }
};

```